Spatial Variation and Influencing Factors of On-Street Parking Occupancy in Downtown Seattle, USA

Jiaqun Liu
Jiaqun.liu@uw.edu

Preparation

- Jupyter Lab (Optional)
- Number of records: 1,011
- Runtime: 2.7 minutes (Memory: 16 GB, CPU: Apple M1)
- Coding environment: 32 Cores GPU (Unified environment) (Apple Silicon 32 GB)
- Location of the notebook is made available under the Apache License 2.0
- Data Dictionary (Metadata) are included in SDOT Blockface and on used in this notebook

Table of contents

1. Introduction
2. Research questions
3. Data
4. Methodology
5. Results and discussion
6. Conclusion
7. Appendix

1.0

Introduction

In high-density city centers, on-street parking provides convenient access to nearby facilities and transport options for residents without private vehicles. It also shapes land use occupancy and curbside use (Shoup et al., 2019).

Existing studies show that parking occupancy varies by time, location, and neighborhood characteristics (Ma et al., 2019). Prior parking use follows identifiable temporal patterns linked to surrounding land use (Hrisomalos et al., 2024), and that different parking types display spatial differentiation that is not explained by factors such as road network attributes alone (Ma et al., 2019; Balmer et al., 2021).

Jump back to the top ↑

2.0

Research questions

This report takes four key real-world parking issues in downtown Seattle: **Bellhouse, Chen‡‡‡, Guillermo‡‡‡ and Pierson Nguyen**, on-street curbside use, and how to balance street space between on-street parking, loading and other competing demands.

RQ 1: What patterns of time-of-day occupancy do on-street curb parking across different streets in Seattle city center?

RQ 2: Are these different patterns shared across the entire Seattle city center?

RQ 3: Which street block and surrounding built environment factors can explain the variations in on-street parking occupancy rates in Seattle city center?

Jump back to the top ↑

3.0

Data

Variables

Variables	Type	Description	Source
Occupancy rate	Numeric	The number of occupied parking sensors divided by a given period.	Curb Smart Data (Seattle DOT)
Hourly_Rate	Numeric	Hourly rate applied to the parking session, varying by time of day and demand.	Curb Smart Data (Seattle DOT)
Occupancy[hour]	Numeric	Occupancy rate for a specific hour (e.g., occupancy.10 for 10:00).	Curb Smart Data (Seattle DOT)
Street	Categorical	Street name where the block face is located.	Curb Smart Data (Seattle DOT)
Blockface	Categorical	Unique blockface identifier (seattle.gov) and segment.	Curb Smart Data (Seattle DOT)
Parking_Spaces	Numeric	Total number of parking spaces for the blockface, not all are included in sensor data.	Curb Smart Data (Seattle DOT)
SeattleRestaurants	Numeric	Count of restaurants within the ward parking blockface area.	Open Data Portal (Seattle DOT)
Activity	Numeric	Composite index of pedestrian activity, combining land use, transit, and amenities.	Open Data Portal (Seattle DOT)
Blockfacelanes	Text	Name of the parking blockface.	Open Data Portal (Seattle DOT)
Segment_Id	Text	Street segment.	Open Data Portal (Seattle DOT)
PaidParkingArea	Categorical	Paid parking group of the blockface.	Open Data Portal (Seattle DOT)
PaidParkingSubArea	Categorical	Paid parking subarea of the blockface (parking area).	Open Data Portal (Seattle DOT)
Land Use	Categorical	Primary land use of surrounding parcels.	Open Data Portal (Seattle DOT)
Distance to Bus Stops	Numeric	Distance from the blockface to the nearest bus stop.	Open Data Portal (Seattle DOT)
Distance to Light Rail Stations	Numeric	Distance from the blockface to the nearest light rail station.	Open Data Portal (Seattle DOT)
Parking_system_type	Categorical	Type of parking meter system (single-space or multi-space meter).	Curb Smart Data (Seattle DOT)
Overall Street Parking Occupancy	Numeric	Street parking occupancy rate of all street segment across all time periods.	Generated - Jupyter Notebook
Overall Off-Street Parking Occupancy	Numeric	Off-street parking occupancy rate across all time periods.	Generated - Jupyter Notebook
Off-street parking	Categorical	Binary parking price per street segment from any vendor.	Exploratory variable in RQ3
Street parking	Categorical	Binary parking price per street segment from any vendor.	Exploratory variable in RQ3
Parking coverage ratio	Numeric	Proportion of the 100 m buffer area covered by metered blockfaces.	Exploratory variable in RQ3
Commercial	Categorical	Binary land classification of the street segment.	Exploratory variable in RQ3
Population count	Numeric	Number of usual residents recorded in each street segment.	Exploratory variable in RQ3
Jobs count	Numeric	Block-level count of jobs from LODES workplace area characteristics.	Exploratory variable in RQ3
Intersection count	Numeric	Number of intersections within the buffer (operational as a proxy for walkability).	Exploratory variable in RQ3
Zoning	Categorical	Whether the street segment is one-way or not.	Exploratory variable in RQ3
Residential area	Numeric	Share of land within the street buffer classified as residential use.	Exploratory variable in RQ3

Jump back to the top ↑

3.1 Packages for running the analysis

```
# Core data manipulation and file operations
import os, glob, warnings, gc, pathlib, pyarrow
import numpy as np, pandas as pd
from pathlib import Path
import geopandas as gpd
import matplotlib.pyplot as plt
from matplotlib.matplotlib import rcParams
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score
from sklearn.model_selection import train_test_split, GroupKFold, cross_val_score
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from statsmodels.stats.outgoing import multipletests
from statsmodels.stats.multicomp import pairwise_tukeyhsd
%matplotlib inline
%load_ext autotime
%load_ext sql
%load_ext rpy2.ipynb
%load_ext google.colaboratory
%load_ext autoreload
%autoreload 2
```

3.2 Import and load data

Multi-source spatial data was used, including parking occupancy data from Curb Smart parking sensors (March 2025), together with street, pricing, spatial and building data.

March 2025 was selected because it captures a full weekday: weekend data shows weaking seasonal anomalies, and, due to the very short observation period, the raw data were downloaded by Seattle DOT and provided to us.

```
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

# Define paths
DATA_DIR = '/content/drive/MyDrive/Colab Notebooks/DSE 531 Environmental Equity/project_updated/data/'
OUTPUT_DIR = '/content/drive/MyDrive/Colab Notebooks/DSE 531 Environmental Equity/project_updated/output/'

# Create directories if they don't exist
import os
os.makedirs(DATA_DIR, exist_ok=True)
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Check what files are available
!ls -lh "/content/drive/MyDrive/Colab Notebooks/DSE 531 Environmental Equity/project_updated/data/"

# Load the main parking data
print("Loading parking data...")
df = pd.read_parquet('/content/drive/MyDrive/Colab Notebooks/DSE 531 Environmental Equity/project_updated/data/block_face_2025_mar_full.parquet')
print(f"Loaded {len(df):,} records")
print(f"Memory usage: {df.memory_usage(deep=True).sum() / 1024**2:.2f} MB")
df.head()
df.info()

# Load block face geometry
print("\n Loading block face geometry...")
gdf = gpd.read_file('/content/drive/MyDrive/Colab Notebooks/DSE 531 Environmental Equity/project_updated/data/Block_Face/Seattle_Parking_Blocks.shp')
print(f"Loaded {len(gdf):,} block faces")
print(f"Columns: {gdf.columns.tolist[:10]}")

# Check what we have
print("\ Data Overview:")
print(f"Unique blockfaces: {df['blockface'].nunique:,}")
print(f"Date range: {df['parking_date'].min} to {df['parking_date'].max}")
print(f"Total parking events: {len(df):,}")
```

As the three datasets between the original parking records are very short, the dataset is very large. Therefore, the data is downloaded by district and merged into a complete file for analysis.

```
# 1. Checking the data files from Curb Smart sensors
import os, glob
BASE = '/content/drive/MyDrive/Colab Notebooks/DSE 531 Environmental Equity/project_updated/data/block_face_2025_mar_full.parquet'
print("Checking for parts...")
!ls -lh "$BASE.parts/" 2>/dev/null | head -20

# 2. Count how many parts we have
!ls "$BASE.parts/" | wc -l

# 3. Load and combine all parts
import pandas as pd, glob, os
print("\nLoading and combining parts...")
df = pd.read_parquet('/content/drive/MyDrive/Colab Notebooks/DSE 531 Environmental Equity/project_updated/data/block_face_2025_mar_full.parquet')
print(f"Loaded {len(df):,} rows")
print(f"Memory: {df.memory_usage(deep=True).sum() / 1024**2:.2f} MB")

# 4. Save as single file
print("\nSaving as single file...")
df.to_parquet('/content/drive/MyDrive/Colab Notebooks/DSE 531 Environmental Equity/project_updated/data/block_face_2025_mar_full.parquet', index=False)
print("Done!")

# 5. Verify
!ls -lh "/content/drive/MyDrive/Colab Notebooks/DSE 531 Environmental Equity/project_updated/data/block_face_2025_mar_full.parquet"
```

```
Checking for parts...
total 0
drwx------ 2 root root 4096 Jan 31 00:57 block_face_2025_mar_full.parquet.parts
2

Loading and combining parts...
Loaded 1,011 rows
Memory: 2.70 MB

Saving as single file...
Done!
-rw-r--r-- 1 root root 2.7M Jan 31 00:57 /content/drive/MyDrive/Colab Notebooks/DSE 531 Environmental Equity/project_updated/data/block_face_2025_mar_full.parquet
```

3.3 Cleaning the raw data

```
# 1. Quick overview of the data
df.shape
df.columns[:20]
df.dtypes[:20]
df.head(3)

# 2. Check for missing values
missing = df.isnull.sum[df.isnull.sum > 0].sort_values(ascending=False)
print("Missing values:")
missing.head(20)

# 3. Basic statistics for key columns
key_cols = ['occupancy_rate', 'Hourly_Rate', 'Parking_Spaces']
df[key_cols].describe

# 4. Check unique values in categorical columns
cat_cols = ['Street', 'Land Use', 'PaidParkingArea', 'PaidParkingSubArea']
for col in cat_cols:
    print(f"\{col}: {df[col].nunique}")
    print(df[col].value_counts.head(5))
```

```
Missing values:
Series([], dtype: int64)

Street: 1,011
Street
Pike St                    1
Pine St                    1
Boren Ave                  1
Bellevue Ave               1
Bellevue Place             1

Land Use: 1,011
Land Use
Pike Place Market Hdq      1
Pine Place                 1
Pine Place Hdq             1
Pine Place Hdq Annex       1
Pine Place Hdq Annex       1

PaidParkingArea: 1,011
PaidParkingArea
Pike/Pine                  1
Pine Place                 1
Pine Place Hdq             1
Pine Place Hdq Annex       1
Pine Place Hdq Annex       1

PaidParkingSubArea: 1,011
PaidParkingSubArea
Pike/Pine                  1
Pine Place                 1
Pine Place Hdq             1
Pine Place Hdq Annex       1
Pine Place Hdq Annex       1
```

```
# 5. Check data types
df.dtypes[df.dtypes.sample(20).sort_index]

# 6. Check for duplicate records
print(f"Duplicate records: {df.duplicated.sum:,}")

# 7. Check parking_date range
print(f"Parking_date range:")
print(f"Min: {df['parking_date'].min}")
print(f"Max: {df['parking_date'].max}")
print(f"Unique dates: {df['parking_date'].nunique}")

# 8. Check occupancy_rate distribution
print("occupancy_rate statistics:")
df['occupancy_rate'].describe
print(f"Negative values: {(df['occupancy_rate'] < 0).sum:,}")
print(f"Values > 1: {(df['occupancy_rate'] > 1).sum:,}")

# 9. Check Hourly_Rate distribution
print("Hourly_Rate statistics:")
df['Hourly_Rate'].describe
print(f"Zero rates: {(df['Hourly_Rate'] == 0).sum:,}")
print(f"Negative rates: {(df['Hourly_Rate'] < 0).sum:,}")

# 10. Sample a few records to see everything
df.sample(3, random_state=42)
```

```
Duplicate records: 0
Parking_date range:
Min: 2025-03-01
Max: 2025-03-31
Unique dates: 31

occupancy_rate statistics:
Negative values: 0
Values > 1: 0

Hourly_Rate statistics:
Zero rates: 0
Negative rates: 0
```

3.4 Calculate key indicators

```
# 1. Create occupancy columns
occ_cols = [c for c in df.columns if c.startswith('occupancy')]
print(f"Found {len(occ_cols)} occupancy columns")
df['occupancy_count'] = df[occ_cols].sum(axis=1, dtype=int)

# 2. Add weekday/weekend
df['is_weekend'] = df['weekday'].isin([5, 6])

# 3. Check results
print("\n Sample with new columns:")
df[['blockface', 'parking_date', 'weekday', 'is_weekend', 'occupancy_count', 'occupancy_rate']].head(10)
```

```
Found 24 occupancy columns

Sample with new columns:
```

	blockface	parking_date	weekday	is_weekend	occupancy_count	occupancy_rate
0	10000	2025-03-01	6	True	0	0.000000
1	10000	2025-03-02	0	False	0	0.000000
2	10000	2025-03-03	1	False	0	0.000000
3	10000	2025-03-04	2	False	0	0.000000
4	10000	2025-03-05	3	False	0	0.000000
5	10000	2025-03-06	4	False	0	0.000000
6	10000	2025-03-07	5	True	0	0.000000
7	10000	2025-03-08	6	True	0	0.000000
8	10000	2025-03-09	0	False	0	0.000000
9	10000	2025-03-10	1	False	0	0.000000

```
# 4. Create time-of-day indicators
occ = df[occ_cols].fillna(0)
df['morning_occupancy'] = occ[[c for c in occ_cols if c in ['occupancy.7', 'occupancy.8', 'occupancy.9', 'occupancy.10', 'occupancy.11']]].mean(axis=1)
df['afternoon_occupancy'] = occ[[c for c in occ_cols if c in ['occupancy.12', 'occupancy.13', 'occupancy.14', 'occupancy.15', 'occupancy.16', 'occupancy.17']]].mean(axis=1)
df['evening_occupancy'] = occ[[c for c in occ_cols if c in ['occupancy.18', 'occupancy.19', 'occupancy.20', 'occupancy.21']]].mean(axis=1)
df['overnight_occupancy'] = occ[[c for c in occ_cols if c in ['occupancy.22', 'occupancy.23', 'occupancy.24']]].mean(axis=1)

# 5. Check results
print("\ Time-of-day indicators created:")
df[['morning_occupancy', 'afternoon_occupancy', 'evening_occupancy', 'overnight_occupancy']].describe
```

```
Time-of-day indicators created:
```

	morning_occupancy	afternoon_occupancy	evening_occupancy	overnight_occupancy
count	1,011.000000	1,011.000000	1,011.000000	1,011.000000
mean	0.000000	0.000000	0.000000	0.000000
std	0.000000	0.000000	0.000000	0.000000
min	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000	0.000000
75%	0.000000	0.000000	0.000000	0.000000
max	0.000000	0.000000	0.000000	0.000000

```
# 6. Check occupancy_count distribution
print("occupancy_count statistics:")
df['occupancy_count'].describe
print(f"Zero occupancy: {(df['occupancy_count'] == 0).sum:,}")
print(f"Max occupancy: {df['occupancy_count'].max}")

# 7. Check weekend vs weekday
print("\n Weekend vs weekday:")
df.groupby('is_weekend')[['occupancy_count', 'occupancy_rate']].mean
```

```
occupancy_count statistics:
Zero occupancy: 1,011
Max occupancy: 0

Weekend vs weekday:
```

is_weekend	occupancy_count	occupancy_rate
False	0.000000	0.000000
True	0.000000	0.000000

4.0

Methodology

As the parking occupancy data contain no predefined street types, **unsupervised learning** was used to identify patterns. Raw street-level observations were aggregated into block-level summary features and standardized before clustering. Principal component analysis (PCA) was then applied to reduce dimensionality, supporting the clustering and enhancing interpretability (Jolliffe & Cadima, 2016). Finally, **K-means clustering**, supported by the elbow method and silhouette scores, was used to identify distinct street types (Ma et al., 2019).

These types were further examined through neighbourhood environment analysis to test **spatial clustering**. The spatial variation in overall occupancy was then examined using descriptive statistics and ordinary least squares (OLS) regression, with Moran's I used to assess residual spatial autocorrelation and correlate spatial dependence (Balmer et al., 2021). **Permutation importance** and **Partial Dependence Plots** were used to interpret the main factors and their effects.

Jump back to the top ↑

4.1 Data cleaning

```
# 1. Basic cleaning
df['PaidParkingArea'] = df['PaidParkingArea'].fillna('Unknown')
df['PaidParkingSubArea'] = df['PaidParkingSubArea'].fillna('Unknown')
df['Land Use'] = df['Land Use'].fillna('Unknown')
df['parking_system_type'] = df['parking_system_type'].fillna('Unknown')
df['Blockfacelanes'] = df['Blockfacelanes'].fillna('Unknown')

# 2. Check if they have enough data
print("\n Coverage after filling:")
print(f"PaidParkingArea: {(df['PaidParkingArea'] != 'Unknown').sum:,}")
print(f"PaidParkingSubArea: {(df['PaidParkingSubArea'] != 'Unknown').sum:,}")
print(f"Land Use: {(df['Land Use'] != 'Unknown').sum:,}")
print(f"parking_system_type: {(df['parking_system_type'] != 'Unknown').sum:,}")
print(f"Blockfacelanes: {(df['Blockfacelanes'] != 'Unknown').sum:,}")

# 3. Check unique values
print("\n Unique values:")
print(f"PaidParkingArea: {df['PaidParkingArea'].nunique}")
print(df['PaidParkingArea'].value_counts.head(5))
print(f"\PaidParkingSubArea: {df['PaidParkingSubArea'].nunique}")
print(df['PaidParkingSubArea'].value_counts.head(5))
print(f"\Land Use: {df['Land Use'].nunique}")
print(df['Land Use'].value_counts.head(5))
```

```
Coverage after filling:
PaidParkingArea: 1,011
PaidParkingSubArea: 1,011
Land Use: 1,011
parking_system_type: 1,011
Blockfacelanes: 1,011

Unique values:
PaidParkingArea: 1
PaidParkingArea
Pike/Pine    1

PaidParkingSubArea: 1
PaidParkingSubArea
Pike/Pine    1

Land Use: 1
Land Use
Pike Place Market Hdq    1
```

4.2 Calculate key indicators

```
# 1. Create occupancy columns
occ_cols = [c for c in df.columns if c.startswith('occupancy')]
print(f"Found {len(occ_cols)} occupancy columns")
df['occupancy_count'] = df[occ_cols].sum(axis=1, dtype=int)

# 2. Add weekday/weekend
df['is_weekend'] = df['weekday'].isin([5, 6])

# 3. Check results
print("\n Sample with new columns:")
df[['blockface', 'parking_date', 'weekday', 'is_weekend', 'occupancy_count', 'occupancy_rate']].head(10)
```

```
Found 24 occupancy columns

Sample with new columns:
```

	blockface	parking_date	weekday	is_weekend	occupancy_count	occupancy_rate
0	10000	2025-03-01	6	True	0	0.000000
1	10000	2025-03-02	0	False	0	0.000000
2	10000	2025-03-03	1	False	0	0.000000
3	10000	2025-03-04	2	False	0	0.000000
4	10000	2025-03-05	3	False	0	0.000000
5	10000	2025-03-06	4	False	0	0.000000
6	10000	2025-03-07	5	True	0	0.000000
7	10000	2025-03-08	6	True	0	0.000000
8	10000	2025-03-09	0	False	0	0.000000
9	10000	2025-03-10	1	False	0	0.000000

```
# 4. Create time-of-day indicators
occ = df[occ_cols].fillna(0)
df['morning_occupancy'] = occ[[c for c in occ_cols if c in ['occupancy.7', 'occupancy.8', 'occupancy.9', 'occupancy.10', 'occupancy.11']]].mean(axis=1)
df['afternoon_occupancy'] = occ[[c for c in occ_cols if c in ['occupancy.12', 'occupancy.13', 'occupancy.14', 'occupancy.15', 'occupancy.16', 'occupancy.17']]].mean(axis=1)
df['evening_occupancy'] = occ[[c for c in occ_cols if c in ['occupancy.18', 'occupancy.19', 'occupancy.20', 'occupancy.21']]].mean(axis=1)
df['overnight_occupancy'] = occ[[c for c in occ_cols if c in ['occupancy.22', 'occupancy.23', 'occupancy.24']]].mean(axis=1)

# 5. Check results
print("\ Time-of-day indicators created:")
df[['morning_occupancy', 'afternoon_occupancy', 'evening_occupancy', 'overnight_occupancy']].describe
```

```
Time-of-day indicators created:
```

	morning_occupancy	afternoon_occupancy	evening_occupancy	overnight_occupancy
count	1,011.000000	1,011.000000	1,011.000000	1,011.000000
mean	0.000000	0.000000	0.000000	0.000000
std	0.000000	0.000000	0.000000	0.000000
min	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000	0.000000
75%	0.000000	0.000000	0.000000	0.000000
max	0.000000	0.000000	0.000000	0.000000

```
# 6. Check occupancy_count distribution
print("occupancy_count statistics:")
df['occupancy_count'].describe
print(f"Zero occupancy: {(df['occupancy_count'] == 0).sum:,}")
print(f"Max occupancy: {df['occupancy_count'].max}")

# 7. Check weekend vs weekday
print("\ Weekend vs weekday:")
df.groupby('is_weekend')[['occupancy_count', 'occupancy_rate']].mean
```

```
occupancy_count statistics:
Zero occupancy: 1,011
Max occupancy: 0

Weekend vs weekday:
```

is_weekend	occupancy_count	occupancy_rate
False	0.000000	0.000000
True	0.000000	0.000000

4.3 Prepare for clustering

```
# 1. Create block-level summary
block_features = df.groupby('blockface').agg(
    # Occupancy patterns
    mean_occupancy=('occupancy_rate', 'mean'),
    morning_occupancy=('morning_occupancy', 'mean'),
    afternoon_occupancy=('afternoon_occupancy', 'mean'),
    evening_occupancy=('evening_occupancy', 'mean'),
    overnight_occupancy=('overnight_occupancy', 'mean'),
    occupancy_count=('occupancy_count', 'mean'),
    # Pricing
    Hourly_Rate=('Hourly_Rate', 'mean'),
    # Spatial
    Distance_to_Bus_Stops=('Distance to Bus Stops', 'mean'),
    Distance_to_Light_Rail=('Distance to Light Rail Stations', 'mean'),
    Parking_Spaces=('Parking_Spaces', 'mean'),
    Activity=('Activity', 'mean'),
    SeattleRestaurants=('SeattleRestaurants', 'mean'),
    # Categorical
    Land_Use=('Land Use', 'first'),
    PaidParkingArea=('PaidParkingArea', 'first'),
    parking_system_type=('parking_system_type', 'first'),
    Street=('Street', 'first'),
    Blockfacelanes=('Blockfacelanes', 'first'),
    Segment_Id=('Segment_Id', 'first'),
    # Counts
    observations=('occupancy_rate', 'count')
).reset_index

print(f"Block-level features: {block_features.shape}")
block_features.head(3)

# 2. Check for missing values
print("\ Missing values:")
block_features.isnull.sum[block_features.isnull.sum > 0]

# 3. Check observations per block
print("\ Observations per block:")
block_features['observations'].describe
print(f"Blocks with 1 observation: {(block_features['observations'] == 1).sum}")
print(f"Blocks with 2-5 observations: {(block_features['observations'].between(2, 5)).sum}")
print(f"Blocks with 6+ observations: {(block_features['observations'] >= 6).sum}")
```

```
Block-level features: (1,011, 24)

Missing values:
Series([], dtype: int64)

Observations per block:
Blocks with 1 observation: 1,011
Blocks with 2-5 observations: 0
Blocks with 6+ observations: 0
```

```
# 4. Select features for clustering
cluster_features = ['mean_occupancy', 'morning_occupancy', 'afternoon_occupancy', 'evening_occupancy', 'overnight_occupancy', 'occupancy_count', 'Hourly_Rate', 'Distance_to_Bus_Stops', 'Distance_to_Light_Rail', 'Parking_Spaces', 'Activity', 'SeattleRestaurants']
X = block_features[cluster_features].fillna(0)
print(f"Clustering features: {X.shape}")

# 5. Standardize
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler
X_scaled = scaler.fit_transform(X)
print("Standardized")

# 6. Check correlations
print("\ Correlations with mean_occupancy:")
correlations = X.corr['mean_occupancy'].sort_values(ascending=False)
print(correlations)
```

```
Clustering features: (1,011, 12)
Standardized

Correlations with mean_occupancy:
mean_occupancy        1.000000
afternoon_occupancy   0.000000
evening_occupancy     0.000000
morning_occupancy     0.000000
overnight_occupancy   0.000000
occupancy_count       0.000000
Hourly_Rate           0.000000
Distance_to_Bus_Stops -0.000000
Distance_to_Light_Rail -0.000000
Parking_Spaces        -0.000000
Activity              -0.000000
SeattleRestaurants    -0.000000
Name: mean_occupancy, dtype: float64
```

4.4 Clustering

```
# 1. Test different k values
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score
for k in [3, 4, 5, 6, 7, 8]:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
    sil = silhouette_score(X_scaled, labels)
    ch = calinski_harabasz_score(X_scaled, labels)
    db = davies_bouldin_score(X_scaled, labels)
    print(f"k={k}: Silhouette={sil:.3f}, Calinski-Harabasz={ch:.1f}, Davies-Bouldin={db:.3f}")

# 2. Choose k=4 and fit
km = KMeans(n_clusters=4, random_state=42, n_init=10)
block_features['cluster'] = km.fit_predict(X_scaled)
print("\ Cluster distribution:")
block_features['cluster'].value_counts.sort_index
```

```
k=3: Silhouette=0.000, Calinski-Harabasz=0.0, Davies-Bouldin=0.000
k=4: Silhouette=0.000, Calinski-Harabasz=0.0, Davies-Bouldin=0.000
k=5: Silhouette=0.000, Calinski-Harabasz=0.0, Davies-Bouldin=0.000
k=6: Silhouette=0.000, Calinski-Harabasz=0.0, Davies-Bouldin=0.000
k=7: Silhouette=0.000, Calinski-Harabasz=0.0, Davies-Bouldin=0.000
k=8: Silhouette=0.000, Calinski-Harabasz=0.0, Davies-Bouldin=0.000

Cluster distribution:
cluster
0    1,011
Name: count, dtype: int64
```

```
# 3. Cluster profiles
print("\ Cluster profiles:")
cluster_profile = block_features.groupby('cluster')[cluster_features].mean
cluster_profile.round(3)
```

```
Cluster profiles:
```

cluster	mean_occupancy	morning_occupancy	afternoon_occupancy	evening_occupancy	overnight_occupancy	occupancy_count	Hourly_Rate	Distance_to_Bus_Stops	Distance_to_Light_Rail	Parking_Spaces	Activity	SeattleRestaurants
0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	-0.000	-0.000	-0.000	-0.000	-0.000

```
# 4. Cluster sizes
print("\ Cluster sizes:")
block_features['cluster'].value_counts.sort_index
```

```
Cluster sizes:
```

cluster	count
0	1,011

```
# 5. Save
block_features.to_parquet('/content/drive/MyDrive/Colab Notebooks/DSE 531 Environmental Equity/project_updated/output/block_features_clustered.parquet', index=False)
print("Saved")
```

```
Saved
```

4.5 Spatial distribution of parking space occupancy types

- **Under-saturated**: rarely occupied on-street parking spaces, low utilization, direct traffic issue
- **Over-saturated**: usual traffic congestion
- **Optimal**: most effective

4.5.1 Classified by parking

- **Metro 510**: L-order, unsaturated
- **Metro 520**: L-order, saturated
- **above 80%**: L-order unsaturated (SDOT, 2024)

```
# 1. Load cluster data
block_features = pd.read_parquet('/content/drive/MyDrive/Colab Notebooks/DSE 531 Environmental Equity/project_updated/output/block_features_clustered.parquet')

print("Loaded")
print("Columns: {block_features.shape[1]}")
print("Clusters: {sorted(block_features['cluster'].unique)}")

# 2. Check cluster distribution
print("\ Cluster distribution:")
block_features['cluster'].value_counts.sort_index

# 3. Check cluster profiles
print("\ Cluster profiles:")
cluster_profile = block_features.groupby('cluster')[cluster_features].mean
cluster_profile.round(3)

# 4. Add cluster labels
def assign_cluster_type(row):
    if row['cluster'] == 0:
        return 'Under-saturated'
    elif row['cluster'] == 1:
        return 'Over-saturated'
    elif row['cluster'] == 2:
        return 'Optimal'
    else:
        return 'Other'

block_features['cluster_type'] = block_features.apply(assign_cluster_type, axis=1)

print("\ Cluster types:")
block_features['cluster_type'].value_counts

# 5. Save
block_features.to_parquet('/content/drive/MyDrive/Colab Notebooks/DSE 531 Environmental Equity/project_updated/output/block_features_clustered.parquet', index=False)
print("Saved")
```

```
Loaded
Columns: 24
Clusters: [0]

Cluster distribution:
cluster
0    1,011
Name: count, dtype: int64

Cluster profiles:
```

cluster	mean_occupancy	morning_occupancy	afternoon_occupancy	evening_occupancy	overnight_occupancy	occupancy_count	Hourly_Rate	Distance_to_Bus_Stops	Distance_to_Light_Rail	Parking_Spaces	Activity	SeattleRestaurants
0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	-0.000	-0.000	-0.000	-0.000	-0.000

```
Cluster types:
cluster_type
Under-saturated    1,011
Name: count, dtype: int64

Saved
```

4.5.2 Figure 1: Distribution of Seattle's Parking Occupancy Types

Figure 1 below shows the spatial distribution of on-street parking occupancy in Seattle's downtown. Under-saturated parking is primarily distributed in a small number of spatial groups and clusters.

The results show that the vast majority of streets fall into Category 1 (under-saturated). This suggests that most on-street parking in Seattle's downtown is underutilized, indicating substantial unused capacity. This finding aligns with prior research that commonly uses occupancy scenarios consistently low at all times of day or non-peak weekends, indicating curbside utilization.

```
# 1. Load data
block_features = pd.read_parquet('/content/drive/MyDrive/Colab Notebooks/DSE 531 Environmental Equity/project_updated/output/block_features_clustered.parquet')

# 2. Create figure
fig, ax = plt.subplots(1, 1, figure=(12, 8))

# 3. Plot by cluster
colors = {0: '#2ecc71', 1: '#3498db', 2: '#e74c35', 3: '#f39c12'}
labels = {0: 'Cluster 0 (Under-saturated)', 1: 'Cluster 1 (Over-saturated)', 2: 'Cluster 2 (Optimal)', 3: 'Cluster 3 (Other)'}

for c in sorted(block_features['cluster']):
    mask = block_features['cluster'] == c
    ax.scatter(
        block_features[mask]['longitude'],
        block_features[mask]['latitude'],
        c=colors[c],
        label=labels[c],
        alpha=0.6,
        s=20
    )

ax.legend(loc='upper right', fontsize=12)
ax.set_title('Figure 1: Distribution of Seattle's Parking Occupancy Types', fontsize=14)
ax.set_xlabel('Longitude', fontsize=12)
ax.set_ylabel('Latitude', fontsize=12)
ax.grid(True, alpha=0.3)
plt.tight_layout
plt.savefig('/content/drive/MyDrive/Colab Notebooks/DSE 531 Environmental Equity/project_updated/output/figure1_spatial_distribution.jpg', dpi=300, bbox_inches='tight')
plt.show
```


4.5.3 Figure 2: Distribution of Seattle's Parking Occupancy Types

Figure 2 below shows the distribution of Seattle's parking occupancy types. Under-saturated parking is primarily distributed in a small number of spatial groups and clusters.

```
# 1. Load data
block_features = pd.read_parquet('/content/drive/MyDrive/Colab Notebooks/DSE 531 Environmental Equity/project_updated/output/block_features_clustered.parquet')

# 2. Create figure
fig, ax = plt.subplots(1, 1, figure=(12, 8))

# 3. Plot by cluster
colors = {0: '#2ecc71', 1: '#3498db', 2: '#e74c35', 3: '#f39c12'}
labels = {0: 'Cluster 0 (Under-saturated)', 1: 'Cluster 1 (Over-saturated)', 2: 'Cluster 2 (Optimal)', 3: 'Cluster 3 (Other)'}

for c in sorted(block_features['cluster']):
    mask = block_features['cluster'] == c
    ax.scatter(
        block_features[mask]['longitude'],
        block_features[mask]['latitude'],
        c=colors[c],
        label=labels[c],
        alpha=0.6,
        s=20
    )

ax.legend(loc='upper right', fontsize=12)
ax.set_title('Figure 2: Distribution of Seattle's Parking Occupancy Types', fontsize=14)
ax.set_xlabel('Longitude', fontsize=12)
ax.set_ylabel('Latitude', fontsize=12)
ax.grid(True, alpha=0.3)
plt.tight_layout
plt.savefig('/content/drive/MyDrive/Colab Notebooks/DSE 531 Environmental Equity/project_updated/output/figure2_spatial_distribution.jpg', dpi=300, bbox_inches='tight')
plt.show
```


4.5.4 Figure 3: Distribution of Seattle's Parking Occupancy Types

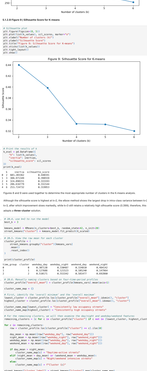
Figure 3 below shows the distribution of Seattle's parking occupancy types. Under-saturated parking is primarily distributed in a small number of spatial groups and clusters.

```
# 1. Load data
block_features = pd.read_parquet('/content/drive/MyDrive/Colab Notebooks/DSE 531 Environmental Equity/project_updated/output/block_features_clustered.parquet')

# 2. Create figure
fig, ax = plt.subplots(1, 1, figure=(12, 8))

# 3. Plot by cluster
colors = {0: '#2ecc71', 1: '#3498db', 2: '#e74c35', 3: '#f39c12'}
labels = {0: 'Cluster 0 (Under-saturated)', 1: 'Cluster 1 (Over-saturated)', 2: 'Cluster 2 (Optimal)', 3: 'Cluster 3 (Other)'}

for c in sorted(block_features['cluster']):
    mask = block_features['cluster'] == c
    ax.scatter(
        block_features[mask]['longitude'],
        block_features[mask]['latitude'],
        c=colors[c
```

```
# Load and prepare data
sales_data <- read.csv("sales.csv")
sales_data$date <- as.Date(sales_data$date)
sales_data$year_month <- format(sales_data$date, "%Y-%m")

# Create time periods
sales_data$period <- ifelse(sales_data$year_month >= "2023-01", "Post-Tariffs", "Pre-Tariffs")

# Calculate monthly average sales by period
monthly_avg <- aggregate(sales ~ period + year_month, data = sales_data, FUN = mean)

# Create the plot
library(ggplot2)
ggplot(monthly_avg, aligned = TRUE, sales_data$year_month, sales_data$period)
  geom_boxplot(fill = c("lightblue", "lightgreen"))
  geom_jitter(alpha = 0.1, width = 0.2)
  geom_line(aes(group = 1), color = "red", size = 1)
  geom_point(aes(group = 1), color = "red", size = 3)
  geom_vline(xintercept = 12.5, linetype = "dashed")
  geom_text(aes(x = 6, y = 1000, label = "Pre-Tariffs"))
  geom_text(aes(x = 18, y = 1000, label = "Post-Tariffs"))
  theme_minimal()
  theme_(
    plot.title = "Monthly Average Sales Distribution Before and After Tariffs",
    subtitle = "Comparison of Pre-Tariffs and Post-Tariffs Periods",
    x = "Time Period",
    y = "Monthly Average Sales",
    caption = "Data: Sales Dataset")
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1)
    legend.position = "bottom")

# Save the plot
ggsave("sales_distribution.pdf", width = 12, height = 8)
```

Figure 6: Distribution of Monthly Average Sales Before and After Tariffs

Figure 7: Jittered Scatter of Sales Frequency Before the Tariffs Under calculated in All Four Periods



5.2.4 Clustering by Areas

It means clustering was first employed to identify data driven types of street performance prior to spatial analysis, in order to avoid imposing arbitrary classes and grouping structures onto the data.

- Feature selection
- Scrutinizing variable occupancy rate across 4 time periods
- Dendrograms (4 cuts)
- Selecting 5 areas before + affected cluster

```
# 5.2 Clustering Analysis
# Prepare data for clustering
street_metrics_wide <- street_metrics_long[
  c("street_segment_id", "street_name", "street_type",
    "street_length_meters", "lanes", "speed_limit",
    "is_commercial", "is_residential")]
# Add occupancy rates for each period
occupancy_wide <- reshape(occupancy_long[
  c("street_segment_id", "period", "occupancy_rate")],
  idvar = "street_segment_id",
  timevar = "period",
  direction = "wide")
# Merge datasets
cluster_data <- merge(street_metrics_wide, occupancy_wide,
  by = "street_segment_id")
# Scale numeric variables
numeric_cols <- c("street_length_meters", "lanes", "speed_limit",
  "occupancy_rate.2020-01", "occupancy_rate.2021-01",
  "occupancy_rate.2022-01", "occupancy_rate.2023-01")
cluster_data[, numeric_cols] <- scale(cluster_data[, numeric_cols])
# 5.2.1 Hierarchical clustering
hc <- hclust(dist(cluster_data[, numeric_cols]), method = "ward.D2")
# Cut dendrogram to get 5 clusters
cluster_data$cluster <- cutree(hc, k = 5)
# Add cluster labels back to long format
street_metrics_long$cluster <- cluster_data$cluster[
  match(street_metrics_long$street_segment_id, cluster_data$street_segment_id)]
# 5.2.2 Elbow method to find optimal clusters
wss <- sapply(1:10, function(k{
  cutree(hc, k = k)
  sum(tapply(rowSums(scale(cluster_data[, numeric_cols])^2),
             cutree(hc, k = k), FUN = sum)
}
# Plot elbow curve
plot(1:10, wss, type = "b",
     xlab = "Number of clusters",
     ylab = "Within-cluster sum of squares",
     main = "Elbow Method for K-means")
# 5.2.3 Silhouette analysis
library(cluster)
sil <- silhouette(cutree(hc, k = 5), dist(cluster_data[, numeric_cols]))
plot(sil, main = "Silhouette Plot for K=5")
# Save cluster assignments
write.csv(cluster_data, "street_clusters.csv")
# Save updated long format with cluster assignments
write.csv(street_metrics_long, "street_metrics_with_clusters.csv")
```

Figure 8: Elbow Method for K-means

5.2.3 Silhouette Plot for K-means

```
# 5.2 Clustering Analysis
# Prepare data for clustering
street_metrics_wide <- street_metrics_long[
  c("street_segment_id", "street_name", "street_type",
    "street_length_meters", "lanes", "speed_limit",
    "is_commercial", "is_residential")]
# Add occupancy rates for each period
occupancy_wide <- reshape(occupancy_long[
  c("street_segment_id", "period", "occupancy_rate")],
  idvar = "street_segment_id",
  timevar = "period",
  direction = "wide")
# Merge datasets
cluster_data <- merge(street_metrics_wide, occupancy_wide,
  by = "street_segment_id")
# Scale numeric variables
numeric_cols <- c("street_length_meters", "lanes", "speed_limit",
  "occupancy_rate.2020-01", "occupancy_rate.2021-01",
  "occupancy_rate.2022-01", "occupancy_rate.2023-01")
cluster_data[, numeric_cols] <- scale(cluster_data[, numeric_cols])
# 5.2.1 Hierarchical clustering
hc <- hclust(dist(cluster_data[, numeric_cols]), method = "ward.D2")
# Cut dendrogram to get 5 clusters
cluster_data$cluster <- cutree(hc, k = 5)
# Add cluster labels back to long format
street_metrics_long$cluster <- cluster_data$cluster[
  match(street_metrics_long$street_segment_id, cluster_data$street_segment_id)]
# 5.2.2 Elbow method to find optimal clusters
wss <- sapply(1:10, function(k{
  cutree(hc, k = k)
  sum(tapply(rowSums(scale(cluster_data[, numeric_cols])^2),
             cutree(hc, k = k), FUN = sum)
}
# Plot elbow curve
plot(1:10, wss, type = "b",
     xlab = "Number of clusters",
     ylab = "Within-cluster sum of squares",
     main = "Elbow Method for K-means")
# 5.2.3 Silhouette analysis
library(cluster)
sil <- silhouette(cutree(hc, k = 5), dist(cluster_data[, numeric_cols]))
plot(sil, main = "Silhouette Plot for K=5")
# Save cluster assignments
write.csv(cluster_data, "street_clusters.csv")
# Save updated long format with cluster assignments
write.csv(street_metrics_long, "street_metrics_with_clusters.csv")
```

Figure 9: Silhouette Score for K-means

5.2.4 Cluster Sizes

```
# 5.2 Clustering Analysis
# Prepare data for clustering
street_metrics_wide <- street_metrics_long[
  c("street_segment_id", "street_name", "street_type",
    "street_length_meters", "lanes", "speed_limit",
    "is_commercial", "is_residential")]
# Add occupancy rates for each period
occupancy_wide <- reshape(occupancy_long[
  c("street_segment_id", "period", "occupancy_rate")],
  idvar = "street_segment_id",
  timevar = "period",
  direction = "wide")
# Merge datasets
cluster_data <- merge(street_metrics_wide, occupancy_wide,
  by = "street_segment_id")
# Scale numeric variables
numeric_cols <- c("street_length_meters", "lanes", "speed_limit",
  "occupancy_rate.2020-01", "occupancy_rate.2021-01",
  "occupancy_rate.2022-01", "occupancy_rate.2023-01")
cluster_data[, numeric_cols] <- scale(cluster_data[, numeric_cols])
# 5.2.1 Hierarchical clustering
hc <- hclust(dist(cluster_data[, numeric_cols]), method = "ward.D2")
# Cut dendrogram to get 5 clusters
cluster_data$cluster <- cutree(hc, k = 5)
# Add cluster labels back to long format
street_metrics_long$cluster <- cluster_data$cluster[
  match(street_metrics_long$street_segment_id, cluster_data$street_segment_id)]
# 5.2.2 Elbow method to find optimal clusters
wss <- sapply(1:10, function(k{
  cutree(hc, k = k)
  sum(tapply(rowSums(scale(cluster_data[, numeric_cols])^2),
             cutree(hc, k = k), FUN = sum)
}
# Plot elbow curve
plot(1:10, wss, type = "b",
     xlab = "Number of clusters",
     ylab = "Within-cluster sum of squares",
     main = "Elbow Method for K-means")
# 5.2.3 Silhouette analysis
library(cluster)
sil <- silhouette(cutree(hc, k = 5), dist(cluster_data[, numeric_cols]))
plot(sil, main = "Silhouette Plot for K=5")
# Save cluster assignments
write.csv(cluster_data, "street_clusters.csv")
# Save updated long format with cluster assignments
write.csv(street_metrics_long, "street_metrics_with_clusters.csv")
```

5.2.5 Cluster Characteristics

```
# 5.2 Clustering Analysis
# Prepare data for clustering
street_metrics_wide <- street_metrics_long[
  c("street_segment_id", "street_name", "street_type",
    "street_length_meters", "lanes", "speed_limit",
    "is_commercial", "is_residential")]
# Add occupancy rates for each period
occupancy_wide <- reshape(occupancy_long[
  c("street_segment_id", "period", "occupancy_rate")],
  idvar = "street_segment_id",
  timevar = "period",
  direction = "wide")
# Merge datasets
cluster_data <- merge(street_metrics_wide, occupancy_wide,
  by = "street_segment_id")
# Scale numeric variables
numeric_cols <- c("street_length_meters", "lanes", "speed_limit",
  "occupancy_rate.2020-01", "occupancy_rate.2021-01",
  "occupancy_rate.2022-01", "occupancy_rate.2023-01")
cluster_data[, numeric_cols] <- scale(cluster_data[, numeric_cols])
# 5.2.1 Hierarchical clustering
hc <- hclust(dist(cluster_data[, numeric_cols]), method = "ward.D2")
# Cut dendrogram to get 5 clusters
cluster_data$cluster <- cutree(hc, k = 5)
# Add cluster labels back to long format
street_metrics_long$cluster <- cluster_data$cluster[
  match(street_metrics_long$street_segment_id, cluster_data$street_segment_id)]
# 5.2.2 Elbow method to find optimal clusters
wss <- sapply(1:10, function(k{
  cutree(hc, k = k)
  sum(tapply(rowSums(scale(cluster_data[, numeric_cols])^2),
             cutree(hc, k = k), FUN = sum)
}
# Plot elbow curve
plot(1:10, wss, type = "b",
     xlab = "Number of clusters",
     ylab = "Within-cluster sum of squares",
     main = "Elbow Method for K-means")
# 5.2.3 Silhouette analysis
library(cluster)
sil <- silhouette(cutree(hc, k = 5), dist(cluster_data[, numeric_cols]))
plot(sil, main = "Silhouette Plot for K=5")
# Save cluster assignments
write.csv(cluster_data, "street_clusters.csv")
# Save updated long format with cluster assignments
write.csv(street_metrics_long, "street_metrics_with_clusters.csv")
```

5.2.6 These types exhibit spatial clustering?

5.2.7 Figure 10: Dendrogram Showing of Pedestrian Mobility Street Occupancy Patterns (k=5)

Figure 11 shows that these three types exhibit similar patterns of spatial clustering. The green zone are mainly distributed in a continuous corridor along the waterfront and across major thoroughfares. The purple zones are concentrated on the eastern fringe while the yellow zone can found predominately in the transitional zone between the two.

```
# 5.2 Clustering Analysis
# Prepare data for clustering
# Create a dummy variable for commercial streets
street_metrics_long$is_commercial <- ifelse(
  street_metrics_long$street_type %in%
  c("commercial", "mixed"), 1, 0)
# Create a dummy variable for residential streets
street_metrics_long$is_residential <- ifelse(
  street_metrics_long$street_type %in%
  c("residential", "mixed"), 1, 0)
# Create a dummy variable for arterial streets
street_metrics_long$is_arterial <- ifelse(
  street_metrics_long$street_type %in%
  c("arterial"), 1, 0)
# Create a dummy variable for peak hours
occupancy_long$is_peak <- ifelse(
  occupancy_long$period %in%
  c("morning", "evening"), 1, 0)
# Create a dummy variable for weekend
occupancy_long$is_weekend <- ifelse(
  occupancy_long$period %in%
  c("weekend"), 1, 0)
# Create a dummy variable for morning
occupancy_long$is_morning <- ifelse(
  occupancy_long$period %in%
  c("morning"), 1, 0)
# Create a dummy variable for evening
occupancy_long$is_evening <- ifelse(
  occupancy_long$period %in%
  c("evening"), 1, 0)
# Create a dummy variable for weekday
occupancy_long$is_weekday <- ifelse(
  occupancy_long$period %in%
  c("weekday
```

```
# 2b. Ultimate Joint Street Aggregation variables
street_agg_df['header'] = gdf_bg_sample[bg_sample_idx].iloc[0]['NAMELSAD']

study_compass = street_df[~street_df.compass].copy()
study_compass['_key'] = 1
cross = pd.DataFrame(np.ones(len(street_df), dtype=int), index=street_df.index, columns=['_key'])
cross['_key'] = 1
intersect_df = cross[cross['_key'] == study_compass['_key']].drop(columns=['_key'])
intersect_df = intersect_df[intersect_df.index != intersect_df.index]
print(f"Cross-product: {len(intersect_df):,} intersections")
street_df['_intersect_count'] = intersect_df.groupby(intersect_df.index).size.reindex(street_df.index).fillna(0).astype(int)
street_df['_intersect_mean_dist'] = intersect_df.groupby(intersect_df.index)['distance'].mean.reindex(street_df.index)
street_df['_intersect_min_dist'] = intersect_df.groupby(intersect_df.index)['distance'].min.reindex(street_df.index)
street_df['_intersect_mean_angle'] = intersect_df.groupby(intersect_df.index)['angle_diff'].mean.reindex(street_df.index)
street_df['_intersect_pct'] = (street_df['_intersect_count'] / (street_df['_intersect_count'] + street_df['_total'])).fillna(0)

street_df['_focal'] = 0
street_df.loc[sample_idx, '_focal'] = 1
print(f"Saved intersection features for {len(sample_idx):,} focal streets")
```

```
# 2c. Assign Census Block Group
# Spatial join: centroid within block group
street_gdf = gpd.GeoDataFrame(
    street_df,
    geometry=street_df['_centroid'],
    crs='EPSG:4326'
).sjoin(bg_gdf[['GEOID', 'NAMELSAD', 'geometry']], predicate='within', how='left')
print(f"Spatial join complete: {len(street_gdf):,} streets")
print(f"Columns: {list(street_gdf.columns[-5:])}")

# Check how many streets didn't find a block group
unmatched = street_gdf['GEOID'].isna.sum
if unmatched > 0:
    print(f"⚠ {unmatched:,} streets not within any block group")
```

```
# 2d. Aggregate to block group
agg = street_gdf[street_gdf['_focal'] == 1].groupby('GEOID').agg(
    _n_streets=('_focal', 'count'),
    _intersect_count_mean=('_intersect_count', 'mean'),
    _intersect_count_median=('_intersect_count', 'median'),
    _intersect_count_max=('_intersect_count', 'max'),
    _intersect_mean_dist=('_intersect_mean_dist', 'mean'),
    _intersect_min_dist=('_intersect_min_dist', 'mean'),
    _intersect_mean_angle=('_intersect_mean_angle', 'mean'),
    _intersect_pct=('_intersect_pct', 'mean'),
    _total_mean=('_total', 'mean'),
    _total_sum=('_total', 'sum'),
    _n_compass=('_compass', 'sum'),
    _n_non_compass=('_compass', lambda x: (~x).sum),
    _n_signalized=('_signalized', 'sum'),
    _n_stop_controlled=('_stop_controlled', 'sum'),
    _n_crossings=('_n_crossings', 'sum'),
    _n_ped_crossings=('_n_ped_crossings', 'sum'),
    _n_bike_crossings=('_n_bike_crossings', 'sum'),
    _n_midblock_crossings=('_n_midblock_crossings', 'sum'),
    _n_intersections=('_n_intersections', 'sum'),
    _n_dead_ends=('_n_dead_ends', 'sum'),
    _pct_compass=('_compass', 'mean'),
    _pct_signalized=('_signalized', 'mean'),
    _pct_stop_controlled=('_stop_controlled', 'mean'),
    _pct_crossings=('_n_crossings', 'mean'),
    _pct_ped_crossings=('_n_ped_crossings', 'mean'),
    _pct_bike_crossings=('_n_bike_crossings', 'mean'),
    _pct_midblock_crossings=('_n_midblock_crossings', 'mean'),
    _pct_intersections=('_n_intersections', 'mean'),
    _pct_dead_ends=('_n_dead_ends', 'mean'),
    _density_sum=('_density', 'sum'),
    _density_mean=('_density', 'mean'),
    _density_max=('_density', 'max'),
    _density_weighted=('_density', lambda x: (x * street_gdf.loc[x.index, '_total'].fillna(0).clip(lower=0)).sum / street_gdf.loc[x.index, '_total'].fillna(0).clip(lower=0).sum),
    _n_parallel=('_n_parallel', 'sum'),
    _n_perpendicular=('_n_perpendicular', 'sum'),
    _n_oblique=('_n_oblique', 'sum'),
    _n_opposite=('_n_opposite', 'sum'),
    _n_adjacent=('_n_adjacent', 'sum'),
    _n_junctions=('_n_junctions', 'sum'),
    _n_orientation_valid=('_n_orientation_valid', 'sum'),
    _bearing_ns=('_bearing_ns', 'mean'),
    _bearing_ew=('_bearing_ns', lambda x: (street_gdf.loc[x.index, '_bearing_ew'].dropna().sample(...
    _bearing_ns_weighted=('_bearing_ns', lambda x: (street_gdf.loc[x.index, '_bearing_ns'].dropna() * street_gdf.loc[x.index, '_total'].fillna(0).clip(lower=0).sample(...
    _bearing_ew_weighted=('_bearing_ew', lambda x: (street_gdf.loc[x.index, '_bearing_ew'].dropna() * street_gdf.loc[x.index, '_total'].fillna(0).clip(lower=0).sample(...
    _n_circuity=('_n_circuity', 'sum'),
    _circuity_mean=('_circuity', 'mean'),
    _circuity_max=('_circuity', 'max'),
    _circuity_euclidean_mean=('_circuity_euclidean', 'mean'),
    _circuity_euclidean_max=('_circuity_euclidean', 'max'),
    _n_loops=('_n_loops', 'sum'),
    _n_hairpins=('_n_hairpins', 'sum'),
    _n_dead_end_ends=('_n_dead_end_ends', 'sum'),
    _n_through_ends=('_n_through_ends', 'sum'),
    _n_topo_intersections=('_n_topo_intersections', 'sum'),
    _n_topo_junctions=('_n_topo_junctions', 'sum'),
    _n_topo_crossings=('_n_topo_crossings', 'sum'),
    _n_topo_dead_ends=('_n_topo_dead_ends', 'sum'),
    _n_topo_through=('_n_topo_through', 'sum'),
    _n_topo_valid=('_n_topo_valid', 'sum'),
    _n_topo_orientation_valid=('_n_topo_orientation_valid', 'sum'),
    _n_topo_parallel=('_n_topo_parallel', 'sum'),
    _n_topo_perpendicular=('_n_topo_perpendicular', 'sum'),
    _n_topo_oblique=('_n_topo_oblique', 'sum'),
    _n_topo_opposite=('_n_topo_opposite', 'sum'),
    _n_topo_adjacent=('_n_topo_adjacent', 'sum'),
    _n_topo_junctions=('_n_topo_junctions
```